

Authenticating Users

 <https://github.com/learn-co-curriculum/python-p4-authenticating-users>  <https://github.com/learn-co-curriculum/python-p4-authenticating-users/issues/new>

Learning Goals

- Define the term "authentication".
- Understand how websites use login to authenticate users.
- Follow REST conventions for handling session data.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

We've covered how cookies can be used to store data in a user's browser.

One of the most common uses of cookies is for login. In this lesson, we'll cover how to use the Flask session to log users in.

Authentication

Nearly every website in the world uses what we like to call the "wristband" pattern. A lot of nightclubs use this pattern as well.

You arrive at the club. The bouncer checks your ID. They put a wristband on your wrist (or stamp your hand). They let you into the club.

If you leave and come back, the bouncer doesn't look at your ID, they just look for your wristband. If you buy a drink, the bartender doesn't need to see your ID, since your wristband proves you're old enough to buy alcohol.

You arrive at [gmail.com](http://mail.google.com)  [_\(http://mail.google.com\)_](http://mail.google.com). You submit your username and password. Google's servers check to see if your credentials are correct. If they are, Google's servers issue a cookie to your browser. If you visit another page on gmail — or anywhere on google.com for that matter — your browser will show the cookie to the server. The server verifies this cookie, and lets you load your inbox.

The term we use to describe this process is **authentication**. When we talk about authentication in our applications, we are describing how our application can **confirm that our users are who they say they are**.

How This Looks in Flask

Let's look at what the simplest possible login system might look like in a Flask API/React application.

The flow will look like this:

- The user navigates to a login form on the React frontend.
- The user enters their username. There is no password (for now).
- The user submits the form, POSTing to `/login` on the Flask backend.
- In the login view we set a cookie on the user's browser by writing their user ID into the session hash.
- Thereafter, the user is logged in. `session['user_id']` will hold their user ID.

Let's write a view to handle our login route. This class will handle `POST` requests to `/login`:

```
class Login(Resource):
```

```
def get(self):  
    ...  
  
def post(self):  
    ...  
  
api.add_resource(Login, '/login')
```

NOTE: Check out the REST module for a reminder on setting up CRUD APIs with Flask RESTful! We left out the imports and whatnot here.

Typically, your `post()` method would look up a user in the database, verify their login credentials, and then store the authenticated user's id in the session:

```
class Login(Resource):  
  
    def post(self):  
        user = User.query.filter(  
            User.username == request.get_json()['username']  
        ).first()  
  
        session['user_id'] = user.id  
        return user.to_dict()
```

There's no way for the server to log you out right now. To log yourself out, you'll have to delete the cookie from your browser.

Here's what the login component might look like on the frontend:

```
function Login({ onLogin }) {  
    const [username, setUsername] = useState("");
```

```
function handleSubmit(e) {
  e.preventDefault();
  fetch("/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ username }),
  })
    .then((r) => r.json())
    .then((user) => onLogin(user));
}

return (
  <form onSubmit={handleSubmit}>
    <input
      type="text"
      value={username}
      onChange={(e) => setUsername(e.target.value)}
    />
    <button type="submit">Login</button>
  </form>
);
}
```

When the user submits the form, they'll be logged in! Our `onLogin` callback function would handle saving the logged in user's details in state.

Staying Logged In

Using the wristband analogy, in the example above, we've shown our ID at the door (`username`) and gotten our wristband (`session['user_id']`) from the backend. So our backend has a means of identifying us with each request using the session object.

Our frontend also knows who we are, because our user data was saved in state after logging in.

What happens now if we leave the club and try to come back in, by refreshing the page on the frontend? Well, our **frontend** doesn't know who we are any more, since we lose our frontend state after refreshing the page. Our **backend** does know who we are though — so we need a way of getting the user data from the backend into state when the page first loads.

Here's how we might accomplish that. First, we need a route to retrieve the user's data from the database using the session hash:

```
class CheckSession(Resource):

    def get(self):
        user = User.query.filter(User.id == session.get('user_id')).first()
        if user:
            return user.to_dict()
        else:
            return {'message': '401: Not Authorized'}, 401

api.add_resource(CheckSession, '/check_session')
```

Then, we can try to log the user in from the frontend as soon as the application loads:

```
function App() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch("/check_session").then((response) => {
      if (response.ok) {
        response.json().then((user) => setUser(user));
      }
    });
  }, []);
}
```

```
if (user) {  
  return <h2>Welcome, {user.username}!</h2>;  
} else {  
  return <Login onLogin={setUser} />;  
}  
}
```

This is the equivalent of letting someone use their wristband to come back into the club.

Logging Out

The log out flow is even simpler. We can add a new route for logging out:

```
class Logout(Resource):  
  
    def delete(self): # just add this line!  
        session['user_id'] = None  
        return {'message': '204: No Content'}, 204  
  
api.add_resource(Logout, '/logout')
```

Here's how that might look in the frontend:

```
function Navbar({ onLogout }) {  
  function handleLogout() {  
    fetch("/logout", {  
      method: "DELETE",  
    }).then(() => onLogout());  
  }  
}
```

```
return (  
  <header>  
    <button onClick={handleLogout}>Logout</button>  
  </header>  
)  
};  
}
```

The `onLogout` callback function would handle removing the information about the user from state.

Cookies, Sessions, and Proxies

Since they can't run on the same port, we normally run our `React` client on `http://localhost:3000` and `Flask` server on `http://localhost:5555`. We've seen how to enable `CORS` in the server to receive requests from other origins (different ports mean different origins). However, by default `CORS` does not allow cookies to be submitted across origins due to potential security implications. Thus, while `CORS` enables the server to accept requests from other origins, the browser is not actually storing the cookies/sessions generated by the server. Since a session is stored as a cookie, this presents a challenge to our ability to authenticate users.

The simplest solution is to use a proxy. Recall a proxy field must be specified in `package.json`:

```
"proxy": "http://localhost:5555"
```

We've seen how a proxy lets us write `fetch` requests in our `React` frontend that don't include the backend domain. Thus, we can pass `"/login"` as the first parameter to `fetch` rather than `"http://localhost:5555/login"`.

```
function Login({ onLogin }) {  
  const [username, setUsername] = useState("");  
  
  function handleSubmit(e) {  
    e.preventDefault();  
    fetch("/login", {  
      ...  
    })  
  }  
}
```

```
} )  
...  
}
```

The proxy also let's us get around the **CORS** cookie issue. Rather than the browser sending a request to the server directly, the proxy acts as a bridge to send the request to **http://localhost:5555/login** . The server sends its response back through the proxy. The proxy makes it appear as if the client request and server response were from the same origin, thus allowing us to use cookies and sessions for user authorization.

Conclusion

At its base, login is very simple: the user provides you with credentials by filling out a form, you verify those credentials and set a token in the **session** . In this example, our token was their user ID. We can also log users out by removing their user ID from the session.

Check For Understanding

Before you move on, make sure you can answer the following questions:

- ▶ 1. *In the login and authentication flow you learned in this lesson for Flask API/React applications, in what two places is authentication information stored?*

 - ▶ 2. *In the login and authentication flow you learned in this lesson, what sequence of events happens if the user refreshes the page?*
-

Resources

- [What is Authentication? - auth0](https://auth0.com/intro-to-iam/what-is-authentication)  [_ \(https://auth0.com/intro-to-iam/what-is-authentication\)](https://auth0.com/intro-to-iam/what-is-authentication)

- [API - Flask: class flask.session](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)  [\(https://flask.palletsprojects.com/en/2.2.x/api/#flask.session\)](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)